GoF Design Pattern 과 Architecture

목적

Adapter, Strategy, Builder, Factory Method 와 같은 정식 GoF 디자인 패턴과 Service, Orchestrator, Provider, Policy, DI, Exception 같은 아키텍처·설계 역할을 구분하고, 각 개념의 책임과 적용 위치를 명확히 정의한다.

1. 핵심 구분

Architecture Style 은 시스템 전체의 Layer, Boundary, Component 와 의존 관계를 어떤 구조로 배치할지 정의하는 상위 수준의 구조 방식이다. Layered, Hexagonal, Clean, Onion, Event-Driven 등이 대표적이다.

Design Approach 는 시스템을 어떤 관점과 원칙으로 분석·모델링할지 결정하는 설계 접근법이다. DDD, OOD, Responsibility-Driven Design 등이 이에 해당한다. MVC2 는 HTTP 요청 처리와 응답 연결을 설명하는 Presentation/Web Architecture Pattern 으로 별도 구분한다.\

Design Role 은 특정 계층이나 객체가 시스템에서 담당하는 책임을 나타내는 역할명이다. 역할 자체가 GoF 패턴인 것은 아니며, 하나의 역할 내부에 여러 디자인 패턴이 적용될 수 있다.

Design Pattern 은 반복적으로 나타나는 소프트웨어 설계 문제에 대한 재사용 가능한 해결 구조이다. GoF 패턴은 생성(Creational), 구조(Structural), 행위(Behavioral)로 분류한다.

2. Software Architecture & Design 분류

분류	대표 예	핵심 질문	설계 결과
Architecture Style	Layered, Hexagonal, Clean, Onion, Event-Driven	시스템 전체의 Component, Layer, Boundary 와 의존성을 어떤 구조로 배치할 것인가?	Layer 구조, Port/Adapter 경계, Dependency Rule, Event 흐름
Design Approach	DDD, OOD, Responsibility-Driven Design, Contract/API-First Design	무엇을 중심으로 어떤 사고방식으로 설계할 것인가?	Domain Model, 객체 책임, 협력 관계, Interface/API Contract
Design Role	Service, Orchestrator, Provider, Policy, DI, Exception	이 객체 또는 Component 가 시스템에서 어떤 책임을 담당하는가?	명확한 책임 분리와 협력 관계
Design Pattern	Adapter, Strategy, Builder, Factory Method, Abstract Factory	반복적으로 발생하는 객체·클래스 설계 문제를 어떤 검증된 구조로 해결할 것인가?	객체 생성·구성·협력·알고리즘 교체 구조

3. Architecture Style

Architecture Style 은 시스템 전체를 어떤 구조와 의존 관계로 구성할 것인가를 결정하는 상위 수준의 설계 방식이다. GoF Design Pattern 이 주로 객체·클래스 수준의 반복 문제를 다루는 것과 달리, Architecture Style 은 계층, 경계, 의존 방향, 배포 및 통신 구조 등 시스템 수준의 구조를 다룬다.

Architecture Style	핵심 질문	주된 특징
Layered Architecture	책임을 계층별로 어떻게 분리할 것인가?	Presentation, Application, Domain/Core, Infrastructure 등의 계층으로 책임 분리
Hexagonal Architecture	Core 를 외부 기술로부터 어떻게 격리할 것인가?	Port 를 Core 경계에 두고 Adapter 가 외부 Framework/API 와 연결
Clean Architecture	의존성이 어떻게 핵심 정책을 향하도록 할 것인가?	안쪽의 Business Rule 이 바깥 Framework 에 의존하지 않도록 Dependency Rule 적용
Onion Architecture	Domain 을 중심으로 의존 계층을 어떻게 둘 것인가?	Domain Model 을 중심에 두고 Application 과 Infrastructure 를 바깥 계층에 배치
Event-Driven Architecture	컴포넌트 간 상태 변화와 비동기 처리를 어떻게 연결할 것인가?	Event 발행/구독을 통해 컴포넌트 간 결합도를 낮추고 비동기 흐름 구성
Modular Monolith	하나의 배포 단위 안에서 모듈 경계를 어떻게 유지할 것인가?	단일 애플리케이션으로 배포하되 기능별 모듈의 책임과 의존 경계를 명확히 유지

4. Design Approach

GoF Design Pattern 이 클래스·객체 수준에서 반복되는 설계 문제의 해결 구조를 제공한다면, Architecture / Design Approach 는 시스템의 더 큰 구조, 책임 경계, 도메인 모델링 방식과 요청 처리 방식을 결정하는 상위 수준의 접근법이다.

구분	접근법	핵심 질문	주된 책임/범위
Presentation Architecture	MVC2	HTTP 요청을 누가 받고, 처리 결과를 어떻게 응답으로 연결하는가?	Controller 중심으로 요청 처리와 Model/Application 호출, View 또는 Response 생성을 분리
Domain Design Approach	DDD	복잡한 비즈니스 Domain 과 규칙을 코드에 어떻게 정확하게 표현하는가?	Ubiquitous Language, Bounded Context, Entity, Value Object, Aggregate, Repository, Domain Service 등을 이용한 Domain 중심 설계

MVC2 와 DDD 의 차이. MVC2 는 주로 Presentation/Web 요청 처리 구조를 설명하고, DDD 는 Domain 을 분석하고 모델링하는 설계 접근법이다. 둘은 경쟁 관계가 아니며 하나의 시스템에서 함께 적용할 수 있다.

분류상 주의. MVC2 와 DDD 는 GoF 23 개 디자인 패턴 표에 포함하지 않는다. MVC2 는 Web/Presentation Architecture Pattern 으로, DDD 는 Domain 중심의 Design Approach 로 별도 분류한다.

5. Design Role

분류	역할/개념	핵심 질문	주된 책임
Application 역할	Service	어떤 Use Case 를 제공하는가?	Application 경계를 제공하고 외부 호출에 사용할 기능을 노출
Workflow 역할	Orchestrator	어떤 순서로 실행하는가?	여러 컴포넌트의 호출 순서와 전체 Workflow 를 조정
Provisioning 역할	Provider	어떤 객체·구성을 제공하는가?	조건에 맞는 객체, 정책 또는

			설정을 선택·제공
Rule 역할	Policy	어떤 규칙을 적용하는가?	도메인·상황별 판단 기준, 제약, 선택 규칙 정의
Dependency 관리	DI	객체들을 어디에서 연결하는가?	의존 객체를 외부에서 생성·선택·주입하고 Object Graph 구성
Error 처리	Exception	어떤 실패가 발생했는가?	오류 유형을 표현하고 계층 사이에 실패 정보를 전달

6. GoF Design Pattern

분류	패턴	핵심 질문	주된 책임
Structural	Adapter	서로 다른 Interface 를 어떻게 연결하는가?	기존/외부 Interface 를 Core 가 기대하는 Interface 로 변환
Behavioral	Strategy	어떤 알고리즘으로 실행하는가?	동일 목적의 알고리즘을 캡슐화하여 교체 가능하게 함
Creational	Builder	복잡한 객체를 어떻게 단계적으로 구성하는가?	복합 객체의 생성 과정을 여러 구성 단계로 분리
Creational	Factory Method	구체 객체 생성을 어떻게 위임하는가?	객체 생성 Interface 를 정의하고 구체 생성 타입의 결정을 분리
Creational	Abstract Factory	관련 객체군을 어떻게 일관되게 생성하는가?	서로 연관된 구현 객체들의 Family 생성을 추상화

주의: 실무에서 흔히 사용하는 'Factory'라는 이름은 그 자체로 GoF 의 단일 패턴명이 아니다. GoF 카탈로그에는 Factory Method 와 Abstract Factory 가 별도로 정의된다.

7. 차이점을 한 문장으로 구분

Service: Use Case 를 외부에 제공한다.

Orchestrator: 여러 컴포넌트가 실행되는 순서를 조정한다.

Adapter: 서로 호환되지 않는 Interface 를 연결한다.

Provider: 현재 필요한 객체나 구성을 선택하여 제공한다.

Policy: 적용할 규칙, 판단 기준과 제약을 정의한다.

Strategy: 선택된 목적을 수행하는 알고리즘을 교체 가능하게 만든다.

Builder: 하나의 복잡한 객체를 단계적으로 조립한다.

Factory Method / Abstract Factory: 구체 객체 또는 관련 객체군의 생성 책임을 추상화한다.

DI: 시스템 전체의 의존 객체를 외부에서 연결하고 주입한다.

Exception: 실패의 종류와 의미를 표현하여 전달한다.

8. 특히 혼동하기 쉬운 관계

Policy vs Strategy

Policy 는 '무엇을 선택하거나 허용할 것인가'를 결정하는 규칙이고, Strategy 는 선택된 목적을 '어떤 알고리즘으로 수행할 것인가'를 담당한다. 예: RetrievalPolicy 가 HYBRID 를 선택하면 HybridRetriever 가 실제 검색을 수행한다.

Builder vs Factory

Factory 는 객체 생성 타입과 생성 책임을 캡슐화하는 데 초점이 있고, Builder 는 많은 설정과 부품이 필요한 복잡한 객체를 단계적으로 구성하는 데 초점이 있다.

Provider vs Factory

Provider 의 관심사는 '필요한 것을 제공하는 것'이다. 제공 과정에서 기존 인스턴스를 반환할 수도, Factory 를 이용해 새 객체를 만들 수도 있다. 따라서 Provider 는 역할이고 Factory Method/Abstract Factory 는 생성 패턴이다.

Builder vs Orchestrator

Builder 는 실행 전에 객체를 구성한다. Orchestrator 는 이미 준비된 객체들을 사용하여 업무 흐름을 실행한다.

Service vs Orchestrator

Service 는 Application 이 외부에 제공하는 Use Case 경계이고, Orchestrator 는 그 Use Case 내부의 복잡한 실행 흐름을 조정한다. 단순한 Use Case 에서는 Service 가 직접 조정하여 별도 Orchestrator 가 필요 없을 수도 있다.

DI vs Builder

DI 는 시스템 수준에서 의존 관계와 Object Graph 를 구성한다. Builder 는 특정 복합 객체 하나의 단계적 구성에 집중한다.

Exception vs Policy

Exception 은 '무엇이 실패했는가'를 표현한다. Policy 는 실패 후 재시도, 중단, Fallback 등 '어떻게 대응할 것인가'의 기준을 정의할 수 있다.

9. 적용 원칙

- 패턴 이름을 먼저 정하고 클래스를 만들지 않는다. 실제 변경 가능성과 책임 분리가 필요한 지점에만 적용한다.
- Core 는 LangChain, LangGraph, FastAPI 같은 외부 Framework 를 직접 알지 않도록 한다.
- Policy 와 Strategy 를 분리하여 '판단 규칙의 변경'과 '알고리즘의 변경'이 서로 영향을 최소화하도록 한다.
- Provider, Builder, Factory 를 중복 도입하지 않는다. 객체 생성이 단순하면 DI 만으로 충분할 수 있다.
- Service 와 Orchestrator 를 항상 둘 필요는 없다. Use Case 가 복잡해져 여러 컴포넌트 조정이 명확한 책임이 될 때 Orchestrator 를 분리한다.
- Exception 에는 실패의 의미를 담고, HTTP 상태 코드 같은 전달 기술의 의미는 Interface 계층에서 변환한다.

10. 참고 자료

- Refactoring.Guru, Design Patterns Catalog — GoF 계열 패턴의 생성/구조/행위 분류와 Adapter, Builder, Strategy 등의 설명.
- Martin Fowler, Patterns of Enterprise Application Architecture — Service Layer: 애플리케이션 경계와 제공 연산을 서비스 계층으로 정의.
- Martin Fowler, Layering Principles — 계층 간 낮은 결합도, 관심사 분리, 상위 소비자에 대한 비의존성 등 계층화 원칙.